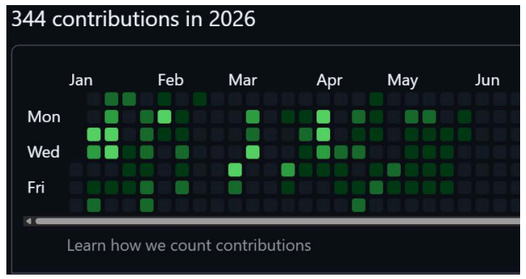
13장 프로젝트 관리 및 협업

13.1 개발 방법론과 일정관리

본 프로젝트는 요구사항이 자주 바뀔 수 잇으며, 팀 전원이 초보이며 빠른 PoC·데모 산출이 목표인 3인 규모로
사전 설계를 고정하는 워터폴보다 반복·피드백 중심의 애자일 방식이 적합하다고 판단했다.

우리 상황	애자일 이점
잦은 요구사항 변경	스프린트 단위 유연 조정
팀원 모두 초보	짧은 피드백으로 빠른 학습
빠른 PoC/데모 목표	동작 기능 우선(완성도 차후)
3인 팀	무거운 문서·프로세스 배제

< 프로젝트 개발 방향성을 고려한 방식 선정 >

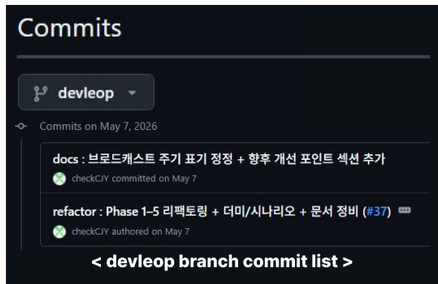


[증빙 13.1] 짧은 스프린트로 반복개발한 기록

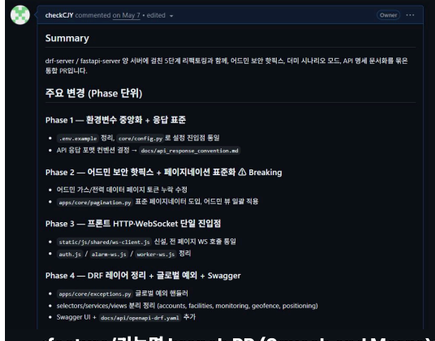
13.2 Git · Github 전략

팀의 Git 운영 경험과 지식이 부족한 상황을 고려하여, Git Flow의 develop/main 브랜치 분리 구조와 GitHub Flow의 가벼운 PR 중심 워크플로를 절충한 브랜치 전략을 적용했다.
기능 단위로 브랜치를 분리하고 PR을 통해 통합함으로써, 동일 코드가 여러 작업자에 의해 중복 수정되는 상황을 줄이고, 개발흐름을 commit으로 파악할 수 있도록 설계하였다.다만 이 전략은 아래와 같은 전제 조건이 있었다.
01 일정 수준 이상의 개발 속도가 전제 되어야 하며
02 코드 취합이 지연될수록 브랜치 간 충돌 위험이 커지는 구조적 단점이 있다.

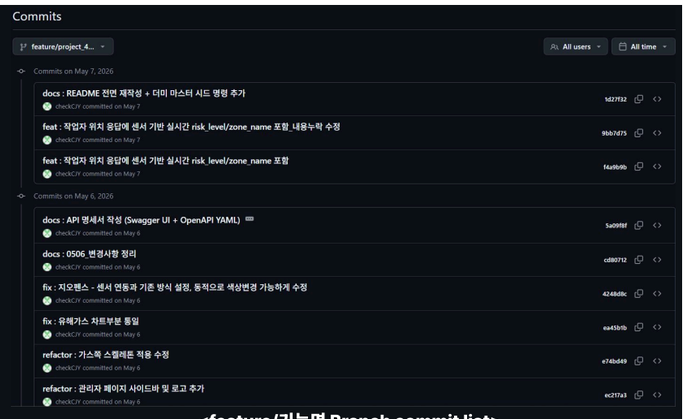
01 답: 이를 보완하기 위해 매일 오후 4시 PR 진행을 권장하고
02 답: develop 브랜치에 Squash and Merge를 수행할 때는 반드시 팀원에게 공유하는 규칙을 채택했다.



< devleop branch commit list >



<feature/기능명 branch PR (Squash and Merge)>



<feature/기능명 Branch commit list>

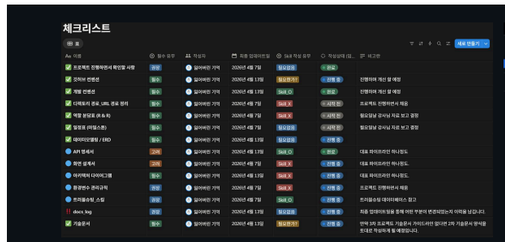
구간	방식	이유
feature→develop	Squash & Merge	커밋 압축
develop→main	Merge commit	해시 보존

<각 branch 별로 병합방식 설정 및 이유>

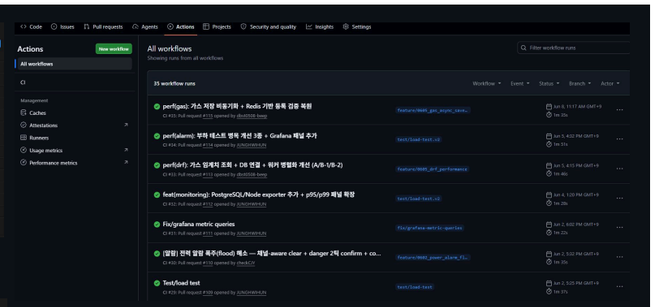
13.3 개발 관리에 관하여

프로젝트 시작 전, 개발과 관련한 문서들을 작성하여 방향 및 향상성을 올리고자 하였다.
팀 전체가 미숙하여 원할하게 개발이 진행되지 않았지만, 아래와 같은 프로세스로 개발을 진행하는걸 배우게 되었다.

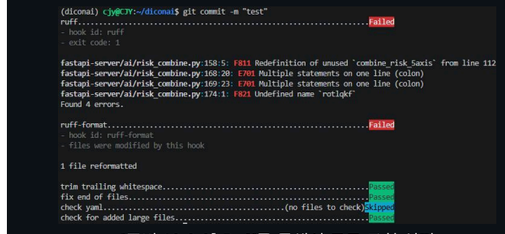
- 01 컨벤션 : 팀원간의 규칙을 설정
- 02 Ruff : 컨벤션을 통한 기본적인 코딩스타일 일관성 유지
- 03 Test : Pytest를 통해서 작성한 기능의 로직의 안정성 확보
- 04 PR : 병합하기 전 버그·성능향상 가능성 확인
- 05 CI : 전체테스트를 자동화를 함 으로써 추후 자동배포시 문제발생 사전에 차단



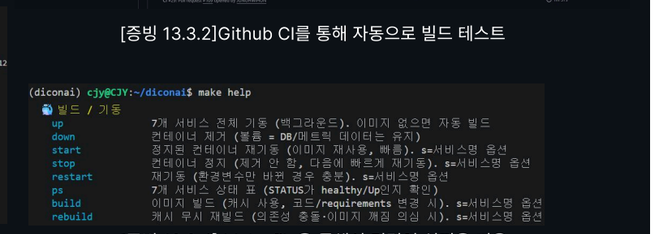
[증빙 13.3.1] 개발 초기당시 작성했던 문서들 모음



[증빙 13.3.2]Github CI를 통해 자동으로 빌드 테스트



[증빙 13.3.3] Ruff를 통해서 코드 규칙 설정



[증빙 13.3.4] Make file을 통해서 명령어 설정을 적용

13.4 협업 · 커뮤니케이션

오프라인이 아닌 온라인 환경에서 개발을 진행하기 위해서
Github 를 통해서 코드공유를 진행하였고, Discord를 이용해 온라인 환경에서 문서공유 및 음성채팅을 진행하였다.

아쉬운 점으로, WBS 또는 Jira를 이용해 기능 완성도 및 완성일자를 파악할 수 있었어야 했다.
다음 스프린로 진행 시 공유해야 할 인프라·신기술 도입시 누락되거나 중복되는 내용을 최소화 할 수 있을 것이다.

원칙	내용
즉시 공유	막히면 혼자 붙잡지 않고 당일 공유
결정 문서화	구두 합의도 Discord·회의록에 기록
비동기 우선	급하지 않으면 메시지 남기고 대기
의견 충돌	당사자 논의 → 안 되면 팀장 조율

< 개발을 진행하면서 팀원간의 규칙 설정>



[증빙 13.4] Discord 진행상황 채널 - 일일 진행현황 / 역할분담 공유